# Curso de R para Ciência Política

Frederico Bertholini (IPOL/UnB)

# Índice

**Apresentação** **5**

**1 Conhecendo o R (Aula 1)** **6**
1.1 O que é o R? . . . 6
1.2 Instalação . . . 6
1.3 O RStudio . . . 6
1.4 O básico: comentários e execução . . . 7
1.5 R como calculadora . . . 7
1.6 Operadores lógicos . . . 8
1.7 Criação de objetos . . . 9
1.8 Classes . . . 10
1.9 Vetores . . . 11
1.9.1 Seleção de elementos . . . 11
1.10 data.frame e tibble . . . 12
1.10.1 Funções para explorar data.frames . . . 13
1.11 Exercícios — Aula 1 . . . 14

**2 Conhecendo seus Dados (Aula 2)** **15**
2.1 Pacotes no R . . . 15
2.2 O pipe `|>` . . . 15
2.3 Importação de dados . . . 16
2.4 Fontes de dados para Ciência Política . . . 16
2.4.1 electionsBR — Dados eleitorais do TSE . . . 17
2.4.2 API da Câmara dos Deputados . . . 17
2.4.3 Outras fontes úteis . . . 17
2.5 Explorando dados: funções essenciais . . . 18
2.6 Nosso dataset de trabalho . . . 18
2.7 Exercícios — Aula 2 . . . 19

**3 Preparando e Manipulando seus Dados (Aula 3)** **20**
3.1 As cinco funções do dplyr . . . 20
3.2 `select()` — escolhendo colunas . . . 20
3.3 `filter()` — filtrando linhas . . . 22
3.4 `mutate()` — criando e modificando colunas . . . 24
3.4.1 Funções úteis dentro de `mutate()` . . . 24
3.5 `summarise()` + `group_by()` . . . 25
3.5.1 `count()` — atalho poderoso . . . 26
3.5.2 Múltiplas variáveis de agrupamento . . . 26
3.6 `arrange()` — ordenando . . . 27
3.7 tidyr: reestruturando dados . . . 27
3.7.1 Dados "tidy" . . . 27
3.7.2 `pivot_wider()` — espalhando valores em colunas . . . 28
3.7.3 `pivot_longer()` — empilhando colunas . . . 28
3.7.4 `separate()` e `unite()` . . . 29

3.8 Joins: juntando tabelas . . . 30
3.8.1 `left_join()` — o join preferencial . . . 30
3.8.2 Outros tipos de join . . . 31
3.9 Limpeza de dados . . . 32
3.9.1 Padronização de nomes com janitor . . . 32
3.9.2 Identificação de duplicatas . . . 32
3.9.3 Valores ausentes . . . 33
3.10 Exercícios — Aula 3 . . . 34

**4 Visualizando e Mapeando seus Dados (Aula 4) 35**
4.1 ggplot2: a gramática dos gráficos . . . 35
4.2 Uma boa exibição gráfica deve… . . . 35
4.3 Recursos para ggplot2 . . . 35
4.4 Gráfico de barras . . . 35
4.4.1 Dispersão: Bases Socioeconômicas do Voto . . . 36
4.5 Histograma . . . 37
4.6 Boxplot com facetas . . . 38
4.7 Heatmap . . . 39
4.8 Escalas . . . 40
4.9 Combinando gráficos com patchwork . . . 41
4.10 Mapas: `sf` + `geobr` . . . 42
4.10.1 Mapa coroplético: votação por estado . . . 42
4.10.2 CRS — Sistema de Referência de Coordenadas . . . 43
4.10.3 Mapa com capitais — pontos sobre polígonos . . . 44
4.11 Pontos Ideais: análise espacial do comportamento político . . . 45
4.11.1 A intuição . . . 45
4.11.2 Simulando votações nominais . . . 45
4.11.3 Estimando pontos ideais via PCA . . . 46
4.11.4 Mapa do Plenário . . . 46
4.11.5 Validação: posição real vs. estimada . . . 47
4.11.6 Com dados reais: API da Câmara . . . 48
4.12 Exercícios — Aula 4 . . . 50

**5 Comunicando seus Dados (Aula 5) 51**
5.1 Quarto: a evolução do R Markdown . . . 51
5.1.1 Estrutura de um documento Quarto . . . 51
5.1.2 Formatos de saída . . . 51
5.1.3 Opções de chunk . . . 52
5.2 Tabelas com `knitr::kable()` . . . 52
5.3 Tabelas de regressão com `modelsummary` . . . 52
5.4 Referências bibliográficas com BibTeX . . . 54
5.5 Reprodutibilidade: regras de ouro . . . 54
5.6 Organização de projetos . . . 55
5.7 Git e GitHub . . . 55
5.8 Próximos passos . . . 55
5.9 Exercícios — Aula 5 . . . 56

**Referências e Recursos** **57**
5.1 Livros e manuais . . . . . 57
5.2 Referências em ciência política . . . . . 57
5.3 Links úteis . . . . . 57

# Apresentação

Esta apostila foi elaborada para acompanhar o **Curso de R** oferecido na Escola de Métodos em Ciências Sociais 2026, promovida pelo Instituto de Ciência Política da Universidade de Brasília. São **15 horas** de curso, distribuídas em **5 encontros** de 3 horas.

O material é autocontido: todo o código pode ser reproduzido em qualquer computador com R ( 4.1) e RStudio instalados. Não é necessário experiência prévia com programação, mas é desejável familiaridade com análise quantitativa em ciências sociais.

**Estrutura do curso:**

- **Aula 1** — Conhecendo o R: o ambiente, a lógica, os objetos fundamentais
- **Aula 2** — Conhecendo seus Dados: importação, pacotes e fontes de dados políticos
- **Aula 3** — Preparando e Manipulando: dplyr, tidyr, joins e limpeza
- **Aula 4** — Visualizando e Mapeando: ggplot2, mapas com sf/geobr, pontos ideais
- **Aula 5** — Comunicando seus Dados: Quarto, tabelas acadêmicas, reprodutibilidade

Como usar esta apostila

Cada seção combina explicação conceitual com código executável. Recomenda-se digitar o código (em vez de copiar e colar) para fixar a sintaxe. Os exercícios ao final de cada aula são fundamentais para consolidar o aprendizado.

---

# 1 Conhecendo o R (Aula 1)

## 1.1 O que é o R?

R é um ambiente de programação voltado para computação estatística e produção gráfica. Criado por Ross Ihaka e Robert Gentleman nos anos 1990, tornou-se um dos softwares mais utilizados por cientistas sociais em todo o mundo, ao lado de Python.

Alguns motivos que fazem do R uma ferramenta valiosa para pesquisa em ciência política:

- **Software livre e gratuito**: sem custo de licença, diferente de SPSS ou Stata
- **Comunidade ativa**: milhares de pacotes criados por pesquisadores de diversas áreas
- **Reprodutibilidade**: o código-fonte é o próprio registro de toda a análise
- **Versatilidade**: dados eleitorais, surveys, text mining, mapas, modelos estatísticos — tudo em um único ambiente
- **Integração com texto**: com Quarto (sucessor do R Markdown), é possível gerar artigos, relatórios e apresentações diretamente do R

> **Um Excel que fez mestrado?**
>
> É uma forma simplificada de pensar. Se algo é possível no Excel, é possível no R. Mas se algo é possível no R — ajustar um modelo multinível, estimar pontos ideais de parlamentares, gerar mapas interativos — provavelmente não é possível no Excel.

## 1.2 Instalação

Para acompanhar o curso, instale:

1. **R** ( 4.1): https://cran.r-project.org/
2. **RStudio Desktop** (gratuito): https://posit.co/download/rstudio-desktop/

Alternativamente, é possível usar o Posit Cloud, que roda no navegador e não exige instalação local.

## 1.3 O RStudio

O RStudio é um ambiente de desenvolvimento integrado (IDE) que organiza o trabalho com R em quatro painéis:

- **Script** (superior esquerdo): onde se escreve e salva o código
- **Console** (inferior esquerdo): onde o código é executado e os resultados aparecem

- **Environment** (superior direito): mostra os objetos criados na sessão
- **Files/Plots/Help** (inferior direito): arquivos, gráficos e documentação

A operação mais fundamental é: escrever uma linha no **Script** e pressionar `Ctrl + Enter` (ou `Cmd + Enter` no Mac) para executá-la no Console.

## 1.4 O básico: comentários e execução

O símbolo `#` marca comentários no código. O R ignora tudo que vem depois de `#` na mesma linha. Comentários servem para documentar o código e facilitar sua leitura posterior.

```
# Isto é um comentário - o R ignora esta linha

5 + 5  # o R executa a soma, mas ignora este trecho
```

```
[1] 10
```

## 1.5 R como calculadora

O R funciona como uma calculadora poderosa. Os operadores aritméticos básicos são:

```
5 + 5    # adição
```

```
[1] 10
```

```
5 - 3    # subtração
```

```
[1] 2
```

```
4 * 9    # multiplicação
```

```
[1] 36
```

```
16 / 2   # divisão
```

```
[1] 8
```

Parênteses controlam a ordem das operações:

```r
(5 + 6) * 3  # parênteses primeiro
```

```
[1] 33
```

```r
5 + 6 * 3    # multiplicação primeiro (sem parênteses)
```

```
[1] 23
```

Potenciação e raiz quadrada:

```r
2 ^ 2      # dois ao quadrado
```

```
[1] 4
```

```r
sqrt(36)   # raiz quadrada de 36
```

```
[1] 6
```

## 1.6 Operadores lógicos

Testes lógicos retornam `TRUE` (verdadeiro) ou `FALSE` (falso). São essenciais para filtrar dados:

```r
5 == 5       # igualdade
```

```
[1] TRUE
```

```r
5 != 3       # diferença
```

```
[1] TRUE
```

```r
5 > 3        # maior que
```

```
[1] TRUE
```

```r
5 <= 5       # menor ou igual
```

```
[1] TRUE
```

Valores lógicos também podem ser combinados:

```r
# E (&): ambos precisam ser verdadeiros
(3 == 3) & (4 != 5)
```

```
[1] TRUE
```

```r
# OU (|): basta um ser verdadeiro
(3 != 3) | (4 != 5)
```

```
[1] TRUE
```

Textos também podem ser comparados:

```r
"Python" == "python"   # FALSE - maiúsculas importam
```

```
[1] FALSE
```

```r
"Stata" != "R"         # TRUE
```

```
[1] TRUE
```

## 1.7 Criação de objetos

A "setinha" `<-` atribui valores a nomes. Em R, quase tudo é um **objeto**: números, textos, tabelas, funções.

```r
sorte <- 5
sorte
```

```
[1] 5
```

Algumas regras para nomes de objetos: letras maiúsculas e minúsculas são diferentes (`sorte` `Sorte`); nomes não podem começar com número (`15luck` dá erro); evite nomes de funções existentes (como `c`, `sum`, `data`).

```r
# Objetos podem ser sobrescritos
sorte <- sorte * 2
sorte
```

```
[1] 10
```

## 1.8 Classes

Em R, todo objeto tem uma **classe** que determina como ele se comporta. As três classes mais básicas são:

```r
x <- 42
class(x)       # numeric
```

```
[1] "numeric"
```

```r
vdd <- TRUE
class(vdd)     # logical
```

```
[1] "logical"
```

```r
nome <- "Ciência Política"
class(nome)    # character
```

```
[1] "character"
```

A classe `factor` é especialmente importante para variáveis categóricas (partido, região, cargo):

```r
partido <- factor(c("PT", "PL", "MDB", "PT", "PL"))
levels(partido)
```

```
[1] "MDB" "PL"  "PT"
```

```r
table(partido)
```

```
partido
MDB  PL  PT
  1   2   2
```

## 1.9 Vetores

Vetores são sequências de valores da mesma classe — a estrutura mais fundamental do R. A função `c()` (de "combinar") cria vetores:

```
estados <- c("SP", "RJ", "MG", "BA", "RS")
populacao <- c(46, 17, 21, 15, 11)
length(estados)
```

```
[1] 5
```

Funções úteis para vetores numéricos:

```
sum(populacao)
```

```
[1] 110
```

```
mean(populacao)
```

```
[1] 22
```

```
sd(populacao)
```

```
[1] 13.89244
```

```
range(populacao)
```

```
[1] 11 46
```

### 1.9.1 Seleção de elementos

Colchetes `[ ]` selecionam elementos por posição ou condição:

```
estados[2]                    # segundo elemento
```

```
[1] "RJ"
```

```r
estados[c(1, 3)]              # primeiro e terceiro
```

```
[1] "SP" "MG"
```

```r
populacao[populacao > 15]     # valores maiores que 15
```

```
[1] 46 17 21
```

O operador `%in%` testa pertencimento a um conjunto:

```r
"BA" %in% estados
```

```
[1] TRUE
```

```r
c("DF", "SP") %in% estados
```

```
[1] FALSE  TRUE
```

## 1.10 data.frame e tibble

O `data.frame` é a tabela do R: cada coluna é uma variável, cada linha é uma observação. O `tibble` (do tidyverse) é uma versão moderna com melhor exibição:

```r
pacman::p_load(tidyverse)

df <- tibble(
  uf = c("SP", "RJ", "MG", "BA"),
  populacao_milhoes = c(46, 17, 21, 15),
  regiao = c("Sudeste", "Sudeste", "Sudeste", "Nordeste")
)

df
```

```
# A tibble: 4 x 3
  uf    populacao_milhoes regiao
  <chr>             <dbl> <chr>
1 SP                   46 Sudeste
2 RJ                   17 Sudeste
3 MG                   21 Sudeste
4 BA                   15 Nordeste
```

O operador $ acessa colunas:

```r
df$uf
```

```
[1] "SP" "RJ" "MG" "BA"
```

```r
mean(df$populacao_milhoes)
```

```
[1] 24.75
```

### 1.10.1 Funções para explorar data.frames

```r
glimpse(df)    # estrutura compacta (preferível a str())
```

```
Rows: 4
Columns: 3
$ uf                <chr> "SP", "RJ", "MG", "BA"
$ populacao_milhoes <dbl> 46, 17, 21, 15
$ regiao            <chr> "Sudeste", "Sudeste", "Sudeste", "Nordeste"
```

```r
nrow(df)       # número de linhas
```

```
[1] 4
```

```r
ncol(df)       # número de colunas
```

```
[1] 3
```

```r
names(df)      # nomes das colunas
```

```
[1] "uf"                "populacao_milhoes" "regiao"
```

## 1.11 Exercícios — Aula 1

1. Crie um vetor `partidos` com os nomes de 5 partidos brasileiros. Crie outro vetor `cadeiras` com o tamanho de suas bancadas (pode inventar). Calcule a média e o total de cadeiras.
2. Crie um `tibble` chamado `deputados` com 4 colunas: `nome`, `partido`, `uf` e `votos`. Preencha com 6 linhas (dados fictícios). Use `glimpse()` para ver a estrutura.
3. A partir do tibble `deputados`, selecione apenas os deputados com mais de 100 mil votos usando colchetes (`deputados[deputados$votos > 100000, ]`).

---

# 2 Conhecendo seus Dados (Aula 2)

## 2.1 Pacotes no R

Pacotes são extensões do R que contêm funções e dados para tarefas específicas. A instalação tradicional usa `install.packages("nome")` e a ativação usa `library(nome)`. Neste curso adotamos `pacman::p_load()`, que instala (se necessário) **e** carrega de uma vez:

```
pacman::p_load(
  tidyverse,       # manipulação + visualização
  scales,          # formatação de eixos
  sf, geobr,       # dados geoespaciais
  patchwork,       # combinar gráficos
  leaflet,         # mapas interativos
  janitor,         # limpeza de dados
  readxl, haven,   # importação Excel/Stata/SPSS
  ipeadatar,        # dados do IPEA
  broom, modelsummary,  # tabelas de regressão
  jsonlite, httr2  # acesso a APIs
)
```

O tidyverse

O `tidyverse` é um conjunto de pacotes integrados para ciência de dados: `dplyr` (manipulação), `ggplot2` (visualização), `tidyr` (reestruturação), `readr` (importação), `stringr` (textos), `forcats` (fatores), `purrr` (programação funcional) e `tibble`. Ao carregar `tidyverse`, todos eles são ativados de uma vez.

## 2.2 O pipe |>

O pipe é o operador que transforma a forma como se lê e escreve código em R. Ele passa o resultado da esquerda como **primeiro argumento** da função à direita:

```
x <- c(1, 4, 9, 16)

# Sem pipe (leitura de dentro para fora):
sqrt(sum(x))
```

```
[1] 5.477226
```

```r
# Com pipe (leitura da esquerda para a direita):
x |> sum() |> sqrt()
```

```
[1] 5.477226
```

> **Nota**
>
> Usamos o pipe nativo `|>` do R (disponível a partir da versão 4.1), não o `%>%` do pacote `magrittr`. A funcionalidade é essencialmente a mesma, mas o pipe nativo é mais rápido e não exige pacote adicional.

## 2.3 Importação de dados

No tidyverse, cada formato de arquivo tem sua função de leitura:

| Formato | Função | Pacote |
|---|---|---|
| `.csv` | `read_csv()` | readr |
| `.xlsx` | `read_xlsx()` | readxl |
| `.dta` (Stata) | `read_dta()` | haven |
| `.sav` (SPSS) | `read_sav()` | haven |
| `.rds` | `read_rds()` | readr |

```r
# Exemplos de importação:
dados <- read_csv("dados/eleicoes.csv")
dados <- read_xlsx("dados/municipios.xlsx")
dados <- read_rds("dados/votacoes.rds")
```

> **Caminhos relativos**
>
> Sempre use **caminhos relativos** (ex: `"dados/arquivo.csv"`) em vez de caminhos absolutos (ex: `"/home/fred/projeto/dados/arquivo.csv"`). Isso torna o projeto reprodutível em qualquer computador. O **RProject** (`.Rproj`) define automaticamente o diretório de trabalho.

## 2.4 Fontes de dados para Ciência Política

O R oferece acesso direto a diversas fontes relevantes para pesquisa em ciência política:

### 2.4.1 electionsBR — Dados eleitorais do TSE

```r
pacman::p_load(electionsBR)

# Candidaturas do DF em 2022
candidatos_df <- candidate_fed(year = 2022, uf = "DF")
glimpse(candidatos_df)
```

### 2.4.2 API da Câmara dos Deputados

A Câmara dos Deputados disponibiliza uma API REST com dados de votações, deputados e proposições. Usamos o pacote `httr2` para acessá-la:

```r
# Buscar votações de agosto/2024
url <- "https://dadosabertos.camara.leg.br/api/v2/votacoes"

resp <- request(url) |>
  req_url_query(
    dataInicio = "2024-08-01",
    dataFim = "2024-08-31"
  ) |>
  req_perform()

votacoes <- resp |>
  resp_body_json() |>
  pluck("dados")
```

> **Nota**
>
> APIs (Application Programming Interfaces) são interfaces que permitem acessar dados diretamente de servidores. A API da Câmara é pública e não exige autenticação. A documentação completa está em: https://dadosabertos.camara.leg.br/swagger/api.html

### 2.4.3 Outras fontes úteis

| Fonte | Pacote / URL | Dados |
|---|---|---|
| TSE | `electionsBR` | Eleições, candidaturas, prestação de contas |
| IBGE / SIDRA | `sidrar` | Censo, PNAD, indicadores municipais |

| Fonte | Pacote / URL | Dados |
|---|---|---|
| IPEA | `ipeadatar` | Séries históricas, IDH |
| CEPESPDATA | `cepespR` | Dados eleitorais compilados |
| Câmara/Senado | `httr2` + API | Votações nominais, proposições |
| V-Dem | `vdemdata` | Indicadores de democracia |
| LAPOP | download | Surveys de opinião pública |

## 2.5 Explorando dados: funções essenciais

Ao importar uma base de dados, o primeiro passo é sempre conhecê-la:

```
glimpse(dados)          # estrutura compacta
head(dados, 10)         # primeiras 10 linhas
dim(dados)              # (linhas, colunas)
names(dados)            # nomes das variáveis
n_distinct(dados$uf)    # contagem de valores únicos
summary(dados$idhm)     # estatísticas descritivas
```

## 2.6 Nosso dataset de trabalho

Neste curso, vamos trabalhar com um dataset real de municípios brasileiros contendo variáveis típicas de análise em ciência política. O arquivo `municipios_br.csv` está na pasta `dados/` do projeto e contém informações dos 5.570 municípios do Brasil.

Para ler o arquivo, usamos a função `read_csv()` do pacote `readr` (parte do tidyverse):

```
municipios <- read_csv("dados/municipios_br.csv")
```

O dataset contém as seguintes variáveis: `cod_ibge` (código IBGE do município), `municipio` (nome), `uf` (unidade federativa), `regiao`, `populacao`, `idhm` (Índice de Desenvolvimento Humano Municipal), `pib_per_capita`, `gini` (índice de desigualdade), `perc_votos_gov` (percentual de votos no governo) e `taxa_comparecimento`.

Essas variáveis permitem explorar correlações plausíveis: municípios com IDHM mais alto tendem a votar menos no governo; a região Nordeste apresenta maior apoio governista; PIB per capita varia com distribuição assimétrica.

```
glimpse(municipios)
```

```
Rows: 5,570
Columns: 10
$ cod_ibge            <dbl> 1001169, 1000766, 1000466, 1001118, 1005194, 10014~
$ municipio           <chr> "Município 1169", "Município 766", "Município 466"~
$ uf                  <chr> "SP", "MG", "MG", "SP", "SE", "SP", "RN", "MG", "M~
$ regiao              <chr> "Sudeste", "Sudeste", "Sudeste", "Sudeste", "Norde~
$ populacao           <dbl> 65199, 4192, 22766, 1000, 30509, 2217, 1824, 27554~
$ idhm                <dbl> 0.821, 0.734, 0.681, 0.622, 0.710, 0.744, 0.525, 0~
$ pib_per_capita      <dbl> 47533, 23544, 20882, 14756, 8685, 113900, 29021, 4~
$ gini                <dbl> 0.49, 0.48, 0.44, 0.51, 0.50, 0.50, 0.48, 0.52, 0.~
$ perc_votos_gov      <dbl> 43.1, 47.3, 40.9, 43.7, 65.3, 39.6, 64.2, 41.1, 61~
$ taxa_comparecimento <dbl> 0.82, 0.69, 0.76, 0.75, 0.81, 0.77, 0.88, 0.84, 0.~
```

## 2.7 Exercícios — Aula 2

1. Use `summary()` para examinar a distribuição das variáveis numéricas de `municipios`. Que padrões você observa?
2. Acesse a documentação da API da Câmara dos Deputados (https://dadosabertos.camara.leg.br) e identifique qual endpoint retorna a lista de deputados em exercício.
3. Pesquise um pacote do R que acesse dados de uma fonte relevante para seu tema de pesquisa. Instale com `pacman::p_load()` e explore a documentação.

---

# 3 Preparando e Manipulando seus Dados (Aula 3)

## 3.1 As cinco funções do dplyr

O pacote `dplyr` oferece uma gramática consistente para manipulação de dados. Cinco funções cobrem a maioria das operações:

| Função | O que faz |
|---|---|
| `select()` | Seleciona **colunas** |
| `filter()` | Filtra **linhas** por condição |
| `mutate()` | Cria ou modifica **colunas** |
| `summarise()` | Resume dados em estatísticas |
| `arrange()` | Ordena **linhas** |

Todas seguem a mesma estrutura: o primeiro argumento é o data.frame (que recebemos via pipe) e os demais são as operações.

## 3.2 `select()` — escolhendo colunas

```
municipios |>
  select(municipio, uf, regiao, idhm)
```

```
# A tibble: 5,570 x 4
   municipio      uf    regiao    idhm
   <chr>          <chr> <chr>    <dbl>
 1 Município 1169 SP    Sudeste  0.821
 2 Município 766  MG    Sudeste  0.734
 3 Município 466  MG    Sudeste  0.681
 4 Município 1118 SP    Sudeste  0.622
 5 Município 5194 SE    Nordeste 0.71
 6 Município 1434 SP    Sudeste  0.744
 7 Município 4801 RN    Nordeste 0.525
 8 Município 612  MG    Sudeste  0.761
 9 Município 231  MG    Sudeste  0.686
10 Município 927  SP    Sudeste  0.793
# i 5,560 more rows
```

Variações úteis com funções auxiliares:

```r
# Colunas cujo nome começa com "perc"
municipios |>
  select(starts_with("perc"))
```

```
# A tibble: 5,570 x 1
   perc_votos_gov
            <dbl>
 1           43.1
 2           47.3
 3           40.9
 4           43.7
 5           65.3
 6           39.6
 7           64.2
 8           41.1
 9           61.4
10           41.2
# i 5,560 more rows
```

```r
# Todas exceto populacao e gasto
municipios |>
  select(-populacao, -gini)
```

```
# A tibble: 5,570 x 8
   cod_ibge municipio      uf    regiao    idhm pib_per_capita perc_votos_gov
      <dbl> <chr>          <chr> <chr>    <dbl>          <dbl>          <dbl>
 1  1001169 Município 1169 SP    Sudeste  0.821          47533           43.1
 2  1000766 Município 766  MG    Sudeste  0.734          23544           47.3
 3  1000466 Município 466  MG    Sudeste  0.681          20882           40.9
 4  1001118 Município 1118 SP    Sudeste  0.622          14756           43.7
 5  1005194 Município 5194 SE    Nordeste 0.71            8685           65.3
 6  1001434 Município 1434 SP    Sudeste  0.744         113900           39.6
 7  1004801 Município 4801 RN    Nordeste 0.525          29021           64.2
 8  1000612 Município 612  MG    Sudeste  0.761          46312           41.1
 9  1000231 Município 231  MG    Sudeste  0.686          33548           61.4
10  1000927 Município 927  SP    Sudeste  0.793          14217           41.2
# i 5,560 more rows
# i 1 more variable: taxa_comparecimento <dbl>
```

Outras funções auxiliares: `ends_with()`, `contains()`, `matches()` (aceita regex), `where()` (por tipo de coluna).

```
# Apenas colunas numéricas
municipios |>
  select(where(is.numeric))
```

```
# A tibble: 5,570 x 7
   cod_ibge populacao  idhm pib_per_capita  gini perc_votos_gov
      <dbl>     <dbl> <dbl>          <dbl> <dbl>          <dbl>
 1  1001169     65199 0.821          47533  0.49           43.1
 2  1000766      4192 0.734          23544  0.48           47.3
 3  1000466     22766 0.681          20882  0.44           40.9
 4  1001118      1000 0.622          14756  0.51           43.7
 5  1005194     30509 0.71            8685  0.5            65.3
 6  1001434      2217 0.744         113900  0.5            39.6
 7  1004801      1824 0.525          29021  0.48           64.2
 8  1000612     27554 0.761          46312  0.52           41.1
 9  1000231     25567 0.686          33548  0.51           61.4
10  1000927     11001 0.793          14217  0.46           41.2
# i 5,560 more rows
# i 1 more variable: taxa_comparecimento <dbl>
```

## 3.3 `filter()` — filtrando linhas

```
# Municípios do Nordeste com IDH acima de 0.7
municipios |>
  filter(regiao == "Nordeste", idhm > 0.7)
```

```
# A tibble: 125 x 10
   cod_ibge municipio      uf    regiao   populacao  idhm pib_per_capita  gini
      <dbl> <chr>          <chr> <chr>        <dbl> <dbl>          <dbl> <dbl>
 1  1005194 Município 5194 SE    Nordeste     30509 0.71            8685  0.5
 2  1003980 Município 3980 MA    Nordeste      1034 0.741          21735  0.41
 3  1003981 Município 3981 MA    Nordeste      3134 0.734          15524  0.4
 4  1003683 Município 3683 PI    Nordeste      2927 0.709          33912  0.48
 5  1002276 Município 2276 BA    Nordeste     16794 0.723          30738  0.46
 6  1003725 Município 3725 PI    Nordeste      1860 0.717          25242  0.51
 7  1004263 Município 4263 CE    Nordeste      5459 0.732          17706  0.59
```

```
 8  1002134 Município 2134 BA    Nordeste      1000 0.712          19747  0.49
 9  1004275 Município 4275 CE    Nordeste     12188 0.73           28645  0.49
10  1004242 Município 4242 CE    Nordeste     10711 0.774          12669  0.52
# i 115 more rows
# i 2 more variables: perc_votos_gov <dbl>, taxa_comparecimento <dbl>
```

Múltiplas condições separadas por vírgula funcionam como **E** (`&`). Para **OU**, use `|`:

```r
# Sudeste ou Sul
municipios |>
  filter(regiao == "Sudeste" | regiao == "Sul") |>
  head(5)
```

```
# A tibble: 5 x 10
  cod_ibge municipio      uf    regiao  populacao  idhm pib_per_capita  gini
     <dbl> <chr>          <chr> <chr>       <dbl> <dbl>          <dbl> <dbl>
1  1001169 Município 1169 SP    Sudeste     65199 0.821          47533  0.49
2  1000766 Município 766  MG    Sudeste      4192 0.734          23544  0.48
3  1000466 Município 466  MG    Sudeste     22766 0.681          20882  0.44
4  1001118 Município 1118 SP    Sudeste      1000 0.622          14756  0.51
5  1001434 Município 1434 SP    Sudeste      2217 0.744         113900  0.5
# i 2 more variables: perc_votos_gov <dbl>, taxa_comparecimento <dbl>
```

O operador `%in%` simplifica múltiplos testes:

```r
# Três estados específicos
municipios |>
  filter(uf %in% c("SP", "RJ", "MG")) |>
  select(municipio, uf, perc_votos_gov) |>
  head(8)
```

```
# A tibble: 8 x 3
  municipio      uf    perc_votos_gov
  <chr>          <chr>          <dbl>
1 Município 1169 SP              43.1
2 Município 766  MG              47.3
3 Município 466  MG              40.9
4 Município 1118 SP              43.7
5 Município 1434 SP              39.6
6 Município 612  MG              41.1
7 Município 231  MG              61.4
8 Município 927  SP              41.2
```

## 3.4 `mutate()` — criando e modificando colunas

```
municipios |>
  mutate(
    margem_vitoria = abs(perc_votos_gov - 50),
    competitivo = margem_vitoria < 10,
    faixa_idh = case_when(
      idhm < 0.55 ~ "Muito Baixo",
      idhm < 0.70 ~ "Baixo",
      idhm < 0.80 ~ "Médio",
      TRUE       ~ "Alto"
    )
  ) |>
  select(municipio, margem_vitoria, competitivo, faixa_idh) |>
  head(8)
```

```
# A tibble: 8 x 4
  municipio      margem_vitoria competitivo faixa_idh
  <chr>                   <dbl> <lgl>       <chr>
1 Município 1169           6.9  TRUE        Alto
2 Município 766            2.70 TRUE        Médio
3 Município 466            9.1  TRUE        Baixo
4 Município 1118           6.3  TRUE        Baixo
5 Município 5194          15.3  FALSE       Médio
6 Município 1434          10.4  FALSE       Médio
7 Município 4801          14.2  FALSE       Muito Baixo
8 Município 612            8.9  TRUE        Médio
```

A função `case_when()` é a versão tidyverse do `ifelse()` aninhado — muito mais legível quando há múltiplas condições.

### 3.4.1 Funções úteis dentro de `mutate()`

```
municipios |>
  mutate(
    log_pop = log(populacao),
    idhm_padronizado = (idhm - mean(idhm)) / sd(idhm),
    ranking_idhm = min_rank(desc(idhm)),
```

```r
    uf_maiuscula = toupper(uf)
  ) |>
  select(municipio, log_pop, idhm_padronizado, ranking_idhm) |>
  head(6)
```

```
# A tibble: 6 x 4
  municipio      log_pop idhm_padronizado ranking_idhm
  <chr>            <dbl>            <dbl>        <int>
1 Município 1169   11.1            1.62            277
2 Município 766     8.34           0.642          1510
3 Município 466    10.0            0.0489         2724
4 Município 1118    6.91          -0.612          4016
5 Município 5194   10.3            0.374          2035
6 Município 1434    7.70           0.754          1301
```

## 3.5 summarise() + group_by()

`summarise()` reduz o data.frame a uma única linha de resumo. Combinado com `group_by()`, calcula estatísticas por grupo:

```r
municipios |>
  group_by(regiao) |>
  summarise(
    n = n(),
    idhm_medio = mean(idhm) |> round(3),
    idhm_dp = sd(idhm) |> round(3),
    voto_gov_medio = mean(perc_votos_gov) |> round(1),
    comparecimento = mean(taxa_comparecimento) |> round(2)
  )
```

```
# A tibble: 5 x 6
  regiao           n idhm_medio idhm_dp voto_gov_medio comparecimento
  <chr>        <int>      <dbl>   <dbl>          <dbl>          <dbl>
1 Centro-Oeste   606      0.694   0.067           49.4           0.82
2 Nordeste      1794      0.601   0.067           59.1           0.82
3 Norte          311      0.631   0.069           51             0.82
4 Sudeste       1668      0.72    0.069           43.3           0.82
5 Sul           1191      0.733   0.071           42.7           0.82
```

### 3.5.1 `count()` — atalho poderoso

```
municipios |>
  count(regiao, sort = TRUE)
```

```
# A tibble: 5 x 2
  regiao           n
  <chr>        <int>
1 Nordeste      1794
2 Sudeste       1668
3 Sul           1191
4 Centro-Oeste   606
5 Norte          311
```

Equivale a `group_by(regiao) |> summarise(n = n()) |> arrange(desc(n))`.

### 3.5.2 Múltiplas variáveis de agrupamento

```
municipios |>
  mutate(faixa_idh = cut(idhm, breaks = c(0, 0.6, 0.75, 1),
                         labels = c("Baixo","Médio","Alto"))) |>
  group_by(regiao, faixa_idh) |>
  summarise(
    n = n(),
    voto_medio = mean(perc_votos_gov) |> round(1),
    .groups = "drop"
  )
```

```
# A tibble: 15 x 4
   regiao       faixa_idh     n voto_medio
   <chr>        <fct>     <int>      <dbl>
 1 Centro-Oeste Baixo        48       53.7
 2 Centro-Oeste Médio       434       49.6
 3 Centro-Oeste Alto        124       46.8
 4 Nordeste     Baixo       895       60.2
 5 Nordeste     Médio       876       58.1
 6 Nordeste     Alto         23       54
 7 Norte        Baixo       112       52.8
```

```
 8 Norte        Médio       185       50.2
 9 Norte        Alto         14       46.9
10 Sudeste      Baixo        65       45.5
11 Sudeste      Médio      1050       44.4
12 Sudeste      Alto        553       41
13 Sul          Baixo        34       46.1
14 Sul          Médio       685       43.7
15 Sul          Alto        472       41
```

## 3.6 `arrange()` — ordenando

```r
municipios |>
  select(municipio, uf, idhm) |>
  arrange(desc(idhm)) |>
  head(5)
```

```
# A tibble: 5 x 3
  municipio      uf     idhm
  <chr>          <chr> <dbl>
1 Município 2663 PR     0.92
2 Município 327  MG     0.92
3 Município 421  MG     0.92
4 Município 541  MG     0.92
5 Município 2498 PR     0.92
```

## 3.7 tidyr: reestruturando dados

Enquanto o `dplyr` faz recortes e cálculos, o `tidyr` muda o **formato** da tabela. As operações centrais vêm em pares inversos:

- `pivot_longer()` e `pivot_wider()`
- `separate()` e `unite()`
- `nest()` e `unnest()`

### 3.7.1 Dados "tidy"

Um dataset está no formato **tidy** quando: cada variável é uma coluna, cada observação é uma linha e cada valor ocupa uma célula.

### 3.7.2 `pivot_wider()` — espalhando valores em colunas

```
# Votação média por região e faixa de IDH - formato wide
votos_wide <- municipios |>
  mutate(faixa_idh = cut(idhm, breaks = c(0, 0.6, 0.75, 1),
                         labels = c("Baixo","Médio","Alto"))) |>
  group_by(regiao, faixa_idh) |>
  summarise(voto_medio = mean(perc_votos_gov) |> round(1), .groups = "drop") |>
  pivot_wider(names_from = faixa_idh, values_from = voto_medio)

votos_wide
```

```
# A tibble: 5 x 4
  regiao       Baixo Médio  Alto
  <chr>        <dbl> <dbl> <dbl>
1 Centro-Oeste  53.7  49.6  46.8
2 Nordeste      60.2  58.1  54
3 Norte         52.8  50.2  46.9
4 Sudeste       45.5  44.4  41
5 Sul           46.1  43.7  41
```

### 3.7.3 `pivot_longer()` — empilhando colunas

```
votos_wide |>
  pivot_longer(-regiao, names_to = "faixa_idh", values_to = "voto_medio")
```

```
# A tibble: 15 x 3
   regiao       faixa_idh voto_medio
   <chr>        <chr>          <dbl>
 1 Centro-Oeste Baixo           53.7
 2 Centro-Oeste Médio           49.6
 3 Centro-Oeste Alto            46.8
 4 Nordeste     Baixo           60.2
 5 Nordeste     Médio           58.1
 6 Nordeste     Alto            54
 7 Norte        Baixo           52.8
 8 Norte        Médio           50.2
 9 Norte        Alto            46.9
```

```
10 Sudeste      Baixo           45.5
11 Sudeste      Médio           44.4
12 Sudeste      Alto            41
13 Sul          Baixo           46.1
14 Sul          Médio           43.7
15 Sul          Alto            41
```

### 3.7.4 separate() e unite()

```r
# Simulando dados de candidaturas com cargo e UF combinados
candidaturas <- tibble(
  id = 1:6,
  cargo_uf = c("Dep.Federal-SP", "Senador-RJ", "Dep.Estadual-MG",
               "Dep.Federal-BA", "Senador-RS", "Dep.Federal-PE")
)

# Separando em duas colunas
candidaturas |>
  separate(cargo_uf, into = c("cargo", "uf"), sep = "-")
```

```
# A tibble: 6 x 3
     id cargo        uf
  <int> <chr>        <chr>
1     1 Dep.Federal  SP
2     2 Senador      RJ
3     3 Dep.Estadual MG
4     4 Dep.Federal  BA
5     5 Senador      RS
6     6 Dep.Federal  PE
```

A operação inversa:

```r
candidaturas |>
  separate(cargo_uf, into = c("cargo", "uf"), sep = "-") |>
  unite("cargo_uf_novo", cargo, uf, sep = " / ")
```

```
# A tibble: 6 x 2
     id cargo_uf_novo
  <int> <chr>
```

```
1     1 Dep.Federal / SP
2     2 Senador / RJ
3     3 Dep.Estadual / MG
4     4 Dep.Federal / BA
5     5 Senador / RS
6     6 Dep.Federal / PE
```

## 3.8 Joins: juntando tabelas

Dados relacionais são a regra em ciência política: uma tabela com informações de deputados, outra com suas votações, outra com dados dos partidos. Para conectá-las, usamos **joins** — operações que combinam tabelas com base em colunas-chave comuns.

```r
# Duas tabelas com chave comum: "partido"
bancadas <- tibble(
  partido = c("PT", "PL", "MDB", "PSOL", "PP"),
  tamanho = c(68, 99, 44, 12, 50)
)

posicao <- tibble(
  partido = c("PT", "PL", "PSOL", "UNIÃO"),
  campo = c("Governo", "Oposição", "Governo", "Oposição")
)
```

### 3.8.1 `left_join()` — o join preferencial

Mantém **todas** as linhas da tabela da esquerda e adiciona as informações da direita quando houver correspondência:

```r
bancadas |>
  left_join(posicao, by = "partido")
```

```
# A tibble: 5 x 3
  partido tamanho campo
  <chr>     <dbl> <chr>
1 PT           68 Governo
2 PL           99 Oposição
3 MDB          44 <NA>
4 PSOL         12 Governo
5 PP           50 <NA>
```

MDB e PP ficaram com `NA` em `campo` porque não estão na tabela `posicao`. Isso é informativo: indica que esses partidos não foram classificados.

### 3.8.2 Outros tipos de join

```r
# inner_join: apenas correspondências
bancadas |> inner_join(posicao, by = "partido")
```

```
# A tibble: 3 x 3
  partido tamanho campo
  <chr>     <dbl> <chr>
1 PT           68 Governo
2 PL           99 Oposição
3 PSOL         12 Governo
```

```r
# anti_join: linhas de x que NÃO estão em y
bancadas |> anti_join(posicao, by = "partido")
```

```
# A tibble: 2 x 2
  partido tamanho
  <chr>     <dbl>
1 MDB          44
2 PP           50
```

```r
# full_join: tudo de ambos
bancadas |> full_join(posicao, by = "partido")
```

```
# A tibble: 6 x 3
  partido tamanho campo
  <chr>     <dbl> <chr>
1 PT           68 Governo
2 PL           99 Oposição
3 MDB          44 <NA>
4 PSOL         12 Governo
5 PP           50 <NA>
6 UNIÃO        NA Oposição
```

Regra prática

Sempre prefira `left_join()`. Ele preserva todas as observações da tabela principal e evidencia quando não há correspondência (preenchendo com `NA`). Isso é mais seguro do que um `inner_join()`, que silenciosamente descarta observações.

## 3.9 Limpeza de dados

### 3.9.1 Padronização de nomes com janitor

```
# Exemplo com nomes de colunas despadronizados
dados_sujos <- tibble(
  `Nome do Candidato` = c("Ana", "Beto"),
  `% Votos Válidos` = c(52.3, 47.7),
  `UF (Sigla)` = c("SP", "RJ")
)

dados_sujos |>
  janitor::clean_names()
```

```
# A tibble: 2 x 3
  nome_do_candidato percent_votos_validos uf_sigla
  <chr>                             <dbl> <chr>
1 Ana                                52.3 SP
2 Beto                               47.7 RJ
```

### 3.9.2 Identificação de duplicatas

```
# Criar dados com duplicata para exemplo
dados_com_dup <- tibble(
  municipio = c("A", "B", "A", "C"),
  uf = c("SP", "RJ", "SP", "MG"),
  votos = c(100, 200, 100, 300)
)

# Encontrar duplicatas
dados_com_dup |>
  janitor::get_dupes(municipio, uf)
```

```
# A tibble: 2 x 4
  municipio uf    dupe_count votos
  <chr>     <chr>      <int> <dbl>
1 A         SP             2   100
2 A         SP             2   100
```

```r
# Remover duplicatas
dados_com_dup |>
  distinct(municipio, uf, .keep_all = TRUE)
```

```
# A tibble: 3 x 3
  municipio uf    votos
  <chr>     <chr> <dbl>
1 A         SP      100
2 B         RJ      200
3 C         MG      300
```

### 3.9.3 Valores ausentes

```r
# Contar NAs por coluna
municipios |>
  summarise(across(everything(), ~ sum(is.na(.x))))
```

```
# A tibble: 1 x 10
  cod_ibge municipio    uf regiao populacao  idhm pib_per_capita  gini
     <int>     <int> <int>  <int>     <int> <int>          <int> <int>
1        0         0     0      0         0     0              0     0
# i 2 more variables: perc_votos_gov <int>, taxa_comparecimento <int>
```

> **Dados faltantes em ciência política**
>
> Valores ausentes (`NA`) não são necessariamente erros. Em dados eleitorais, por exemplo, candidatos que não declararam patrimônio terão `NA` nessa coluna. A decisão sobre como tratar dados faltantes (excluir, imputar, manter) é uma escolha metodológica que deve ser justificada.

## 3.10 Exercícios — Aula 3

1. A partir de `municipios`, crie uma variável `faixa_idh` (Baixo $< 0.6$, Médio 0.6-0.75, Alto $> 0.75$). Agrupe por `regiao` e `faixa_idh`, calcule o voto médio no governo e o número de municípios. Ordene por voto médio decrescente.

2. Crie uma tabela no formato *wide* que mostre a taxa de comparecimento média por região e UF, com as UFs nas colunas.

3. Crie duas tabelas: uma de partidos com seus campos ideológicos (esquerda/centro/direita) e outra de partidos com número de cadeiras. Faça um `left_join()` e identifique quais partidos ficaram sem classificação.

4. Pesquise e aplique `janitor::tabyl()` para criar uma tabela de frequências cruzadas de `regiao` e a `faixa_idh` criada no exercício 1.

---

# 4 Visualizando e Mapeando seus Dados (Aula 4)

## 4.1 ggplot2: a gramática dos gráficos

O pacote `ggplot2` implementa a "gramática dos gráficos" de Leland Wilkinson: toda visualização é construída pela combinação de camadas. A estrutura básica é:

```
ggplot(dados, aes(x = ..., y = ..., color = ...)) +
  geom_tipo() +
  labs(title = "...", x = "...", y = "...") +
  theme_minimal()
```

Os elementos são: **dados** (de onde vêm as informações), **estéticas** (mapeamento de variáveis a propriedades visuais), **geometrias** (tipo de marca gráfica), **escalas** (ajustes nas estéticas), **temas** (aparência geral) e **facetas** (subgráficos).

## 4.2 Uma boa exibição gráfica deve...

Edward Tufte, referência em visualização de dados, propõe que um bom gráfico deve: mostrar os dados, induzir o observador a pensar na substância (não na metodologia), evitar distorções, tornar grandes conjuntos de dados coerentes, encorajar comparações entre partes dos dados e revelar diferentes níveis de detalhe.

Quatro princípios práticos: miniaturas múltiplas (small multiples), menor diferença efetiva, causalidade (respondendo "comparado com o quê?") e contexto.

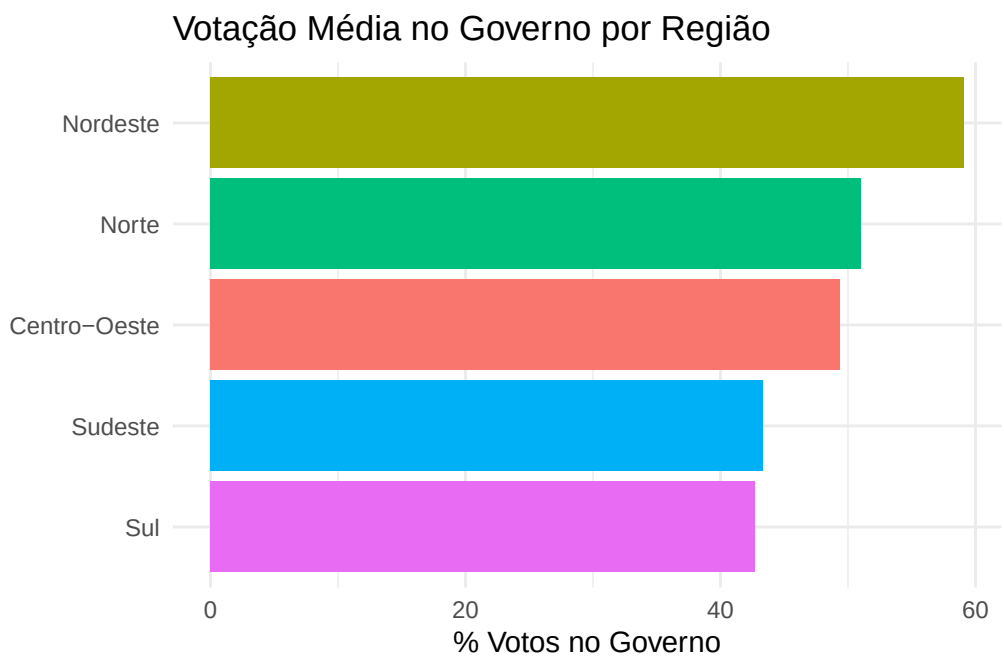
## 4.3 Recursos para ggplot2

Antes de avançar, vale conhecer os recursos de referência:

- R Graphics Cookbook — receitas práticas
- R Graph Gallery — galeria com código
- Fundamentals of Data Visualization — princípios
- Extensões do ggplot2 — pacotes adicionais

## 4.4 Gráfico de barras

```r
municipios |>
  group_by(regiao) |>
  summarise(voto_medio = mean(perc_votos_gov)) |>
  ggplot(aes(x = reorder(regiao, voto_medio), y = voto_medio, fill = regiao)) +
  geom_col() +
  coord_flip() +
  labs(title = "Votação Média no Governo por Região",
       x = "", y = "% Votos no Governo") +
  theme_minimal() +
  theme(legend.position = "none")
```



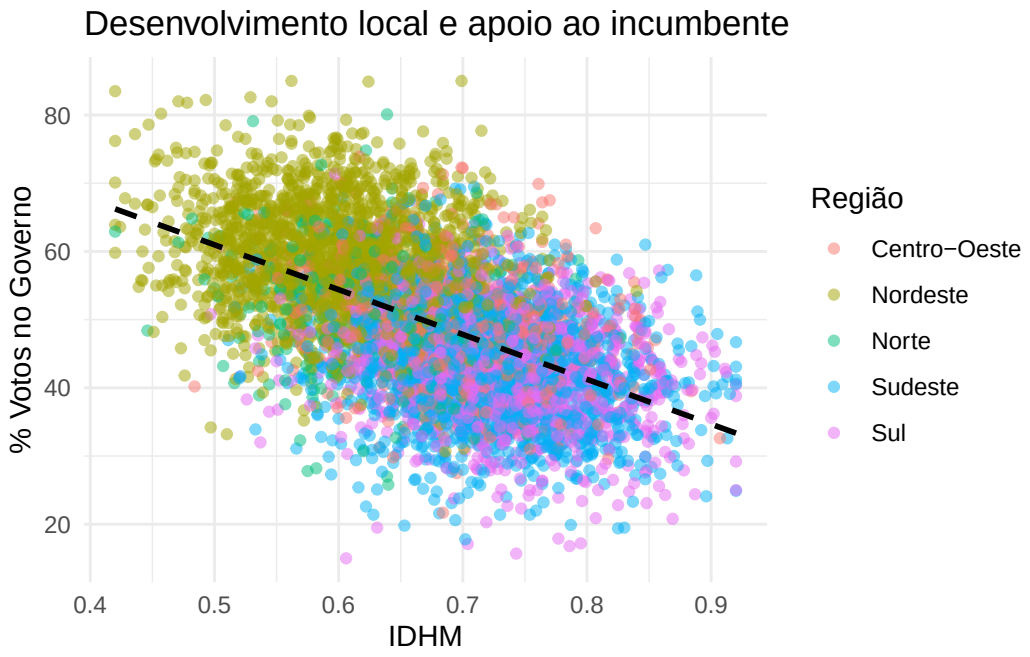

Note o uso de `reorder()` para ordenar as barras pelo valor — muito mais informativo do que ordem alfabética.

### 4.4.1 Dispersão: Bases Socioeconômicas do Voto

A literatura de comportamento político investiga como o desenvolvimento local afeta o apoio a governos incumbentes. Podemos examinar essa relação cruzando o IDHM dos municípios com a proporção de votos no governo.

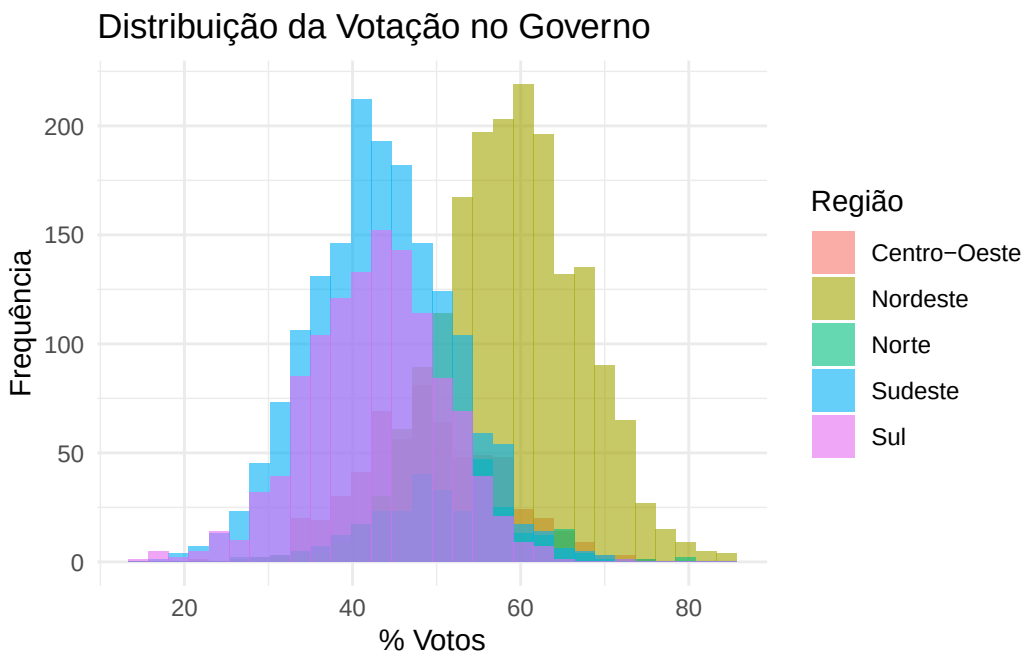
```r
municipios |>
  ggplot(aes(x = idhm, y = perc_votos_gov, color = regiao)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", se = FALSE, color = "black", linetype = "dashed") +
```

```
  labs(title = "Desenvolvimento local e apoio ao incumbente",
       x = "IDHM", y = "% Votos no Governo", color = "Região") +
  theme_minimal()
```




## 4.5 Histograma

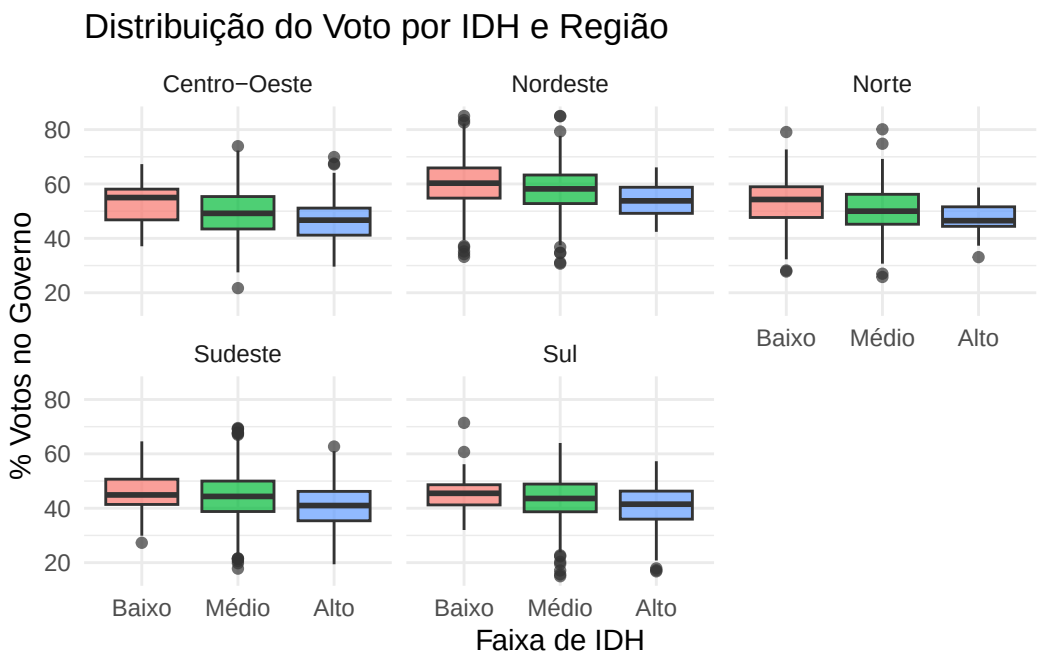
```
municipios |>
  ggplot(aes(x = perc_votos_gov, fill = regiao)) +
  geom_histogram(bins = 30, alpha = 0.6, position = "identity") +
  labs(title = "Distribuição da Votação no Governo",
       x = "% Votos", y = "Frequência", fill = "Região") +
  theme_minimal()
```




## 4.6 Boxplot com facetas

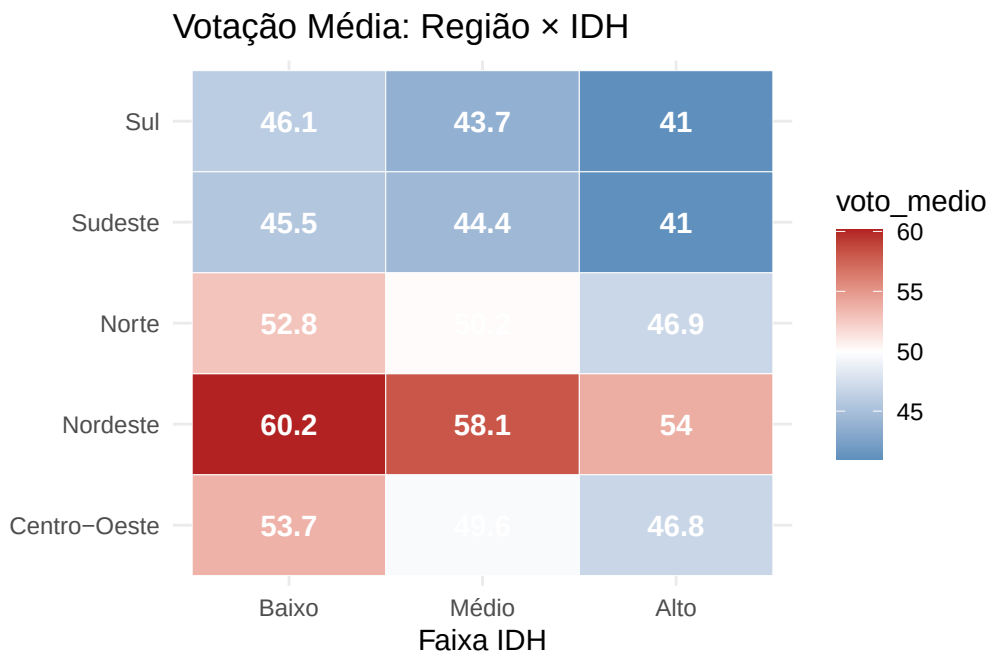
As facetas (`facet_wrap`) permitem criar subgráficos por categorias — o que Tufte chama de "small multiples":

```
municipios |>
  mutate(faixa_idh = cut(idhm, breaks = c(0, 0.6, 0.75, 1),
                         labels = c("Baixo","Médio","Alto"))) |>
  ggplot(aes(x = faixa_idh, y = perc_votos_gov, fill = faixa_idh)) +
  geom_boxplot(alpha = 0.7) +
  facet_wrap(~regiao) +
  labs(title = "Distribuição do Voto por IDH e Região",
       x = "Faixa de IDH", y = "% Votos no Governo") +
  theme_minimal() +
  theme(legend.position = "none")
```




## 4.7 Heatmap

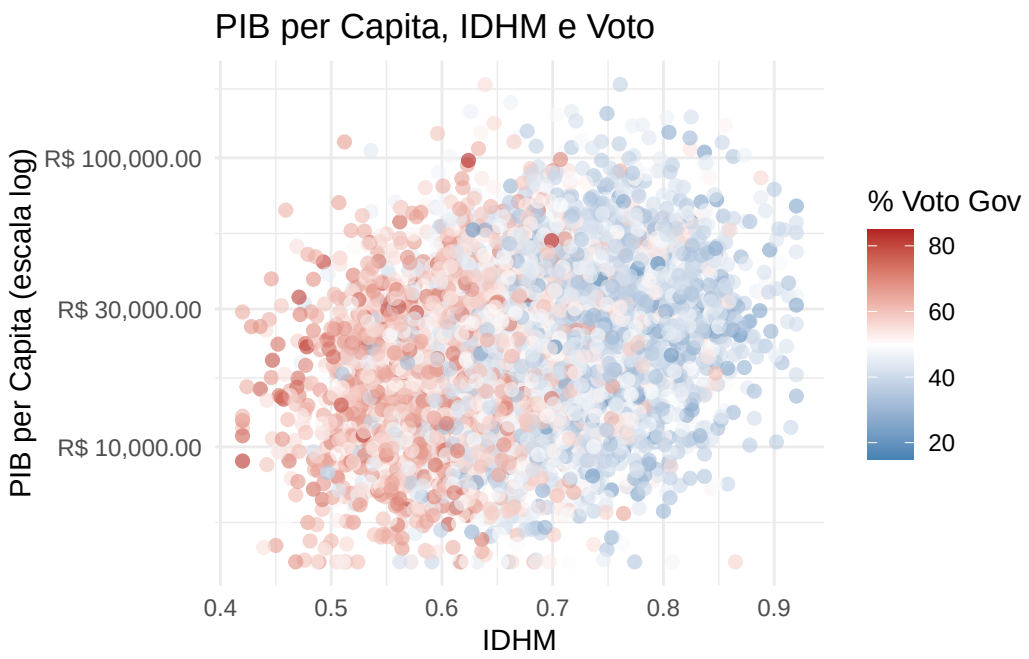
```
municipios |>
  mutate(faixa_idh = cut(idhm, breaks = c(0, 0.6, 0.75, 1),
                         labels = c("Baixo","Médio","Alto"))) |>
  group_by(regiao, faixa_idh) |>
  summarise(voto_medio = mean(perc_votos_gov), .groups = "drop") |>
  ggplot(aes(x = faixa_idh, y = regiao, fill = voto_medio)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(voto_medio, 1)), color = "white", fontface = "bold") +
  scale_fill_gradient2(low = "steelblue", mid = "white", high = "firebrick",
                       midpoint = 50) +
  labs(title = "Votação Média: Região × IDH", x = "Faixa IDH", y = "") +
  theme_minimal()
```




## 4.8 Escalas

Escalas controlam como as variáveis são mapeadas visualmente. Escalas divergentes (com ponto central) são especialmente úteis para dados eleitorais:

```
municipios |>
  ggplot(aes(x = idhm, y = pib_per_capita, color = perc_votos_gov)) +
  geom_point(alpha = 0.6, size = 2) +
  scale_color_gradient2(low = "steelblue", mid = "white", high = "firebrick",
                        midpoint = 50, name = "% Voto Gov") +
  scale_y_log10(labels = scales::dollar_format(prefix = "R$ ")) +
  labs(title = "PIB per Capita, IDHM e Voto",
       x = "IDHM", y = "PIB per Capita (escala log)") +
  theme_minimal()
```



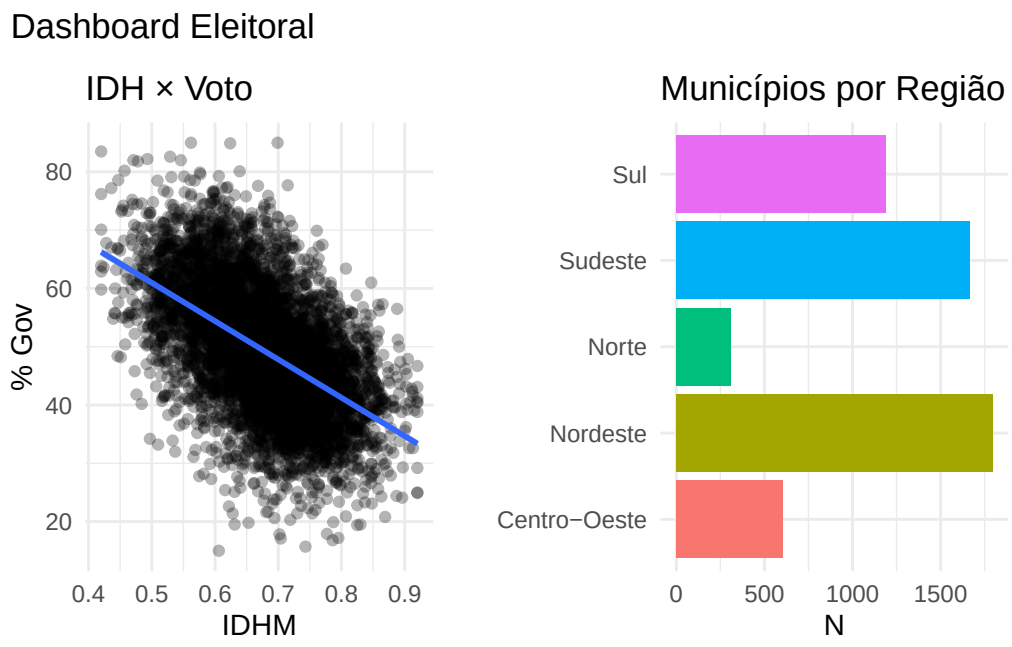

## 4.9 Combinando gráficos com patchwork

O pacote `patchwork` permite combinar gráficos com operadores simples: `+` coloca lado a lado, `/` empilha, `|` separa em colunas.

```
p1 <- municipios |>
  ggplot(aes(x = idhm, y = perc_votos_gov)) +
  geom_point(alpha = 0.3) + geom_smooth(method = "lm") +
  labs(title = "IDH × Voto", x = "IDHM", y = "% Gov") + theme_minimal()

p2 <- municipios |>
  ggplot(aes(x = regiao, fill = regiao)) +
  geom_bar() + coord_flip() +
  labs(title = "Municípios por Região", x = "", y = "N") +
  theme_minimal() + theme(legend.position = "none")

p1 + p2 + plot_annotation(title = "Dashboard Eleitoral")
```




## 4.10 Mapas: `sf` + `geobr`

O pacote `sf` (Simple Features) é o padrão para dados geoespaciais no R. Ele trata mapas como data.frames com uma coluna especial de geometria. O pacote `geobr` disponibiliza as malhas geográficas oficiais do IBGE.

```
# Baixar malha dos estados brasileiros
estados_br <- read_state(year = 2020, showProgress = FALSE)
```

```
# Verificar a estrutura
glimpse(estados_br)
```

```
Rows: 27
Columns: 6
$ code_state   <dbl> 11, 12, 13, 14, 15, 16, 17, 21, 22, 23, 24, 25, 26, 27, 2~
$ abbrev_state <chr> "RO", "AC", "AM", "RR", "PA", "AP", "TO", "MA", "PI", "CE~
$ name_state   <chr> "Rondônia", "Acre", "Amazônas", "Roraima", "Pará", "Amapá~
$ code_region  <dbl> 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, ~
$ name_region  <chr> "Norte", "Norte", "Norte", "Norte", "Norte", "Norte", "No~
$ geom         <MULTIPOLYGON [°]> MULTIPOLYGON (((-65.3815 -1..., MULTIPOLYGON~
```

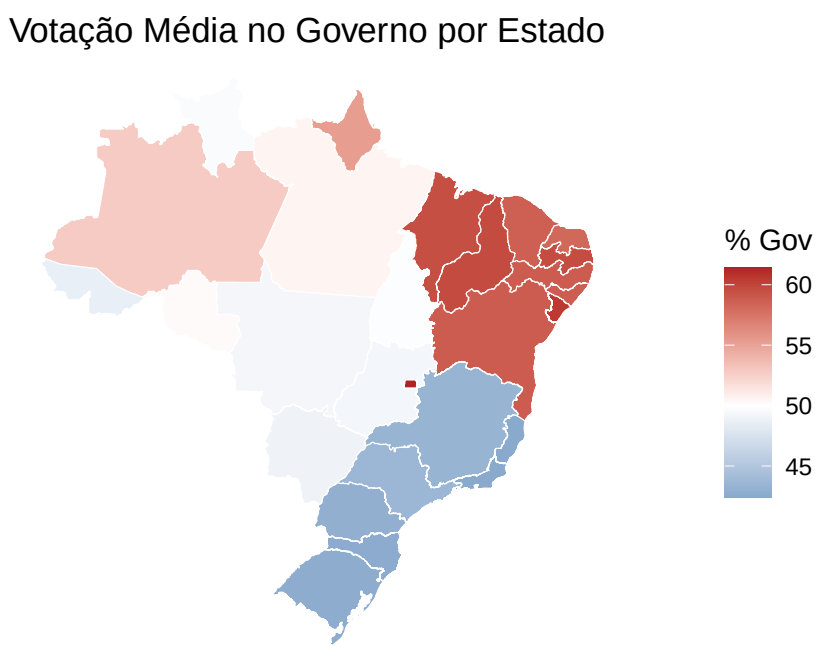
### 4.10.1 Mapa coroplético: votação por estado

Para criar um mapa temático, basta fazer um `left_join` entre a malha geográfica e seus dados:

```r
dados_uf <- municipios |>
  group_by(uf) |>
  summarise(voto_medio = mean(perc_votos_gov), .groups = "drop")

mapa <- estados_br |>
  left_join(dados_uf, by = c("abbrev_state" = "uf"))

ggplot(mapa) +
  geom_sf(aes(fill = voto_medio), color = "white", linewidth = 0.2) +
  scale_fill_gradient2(low = "steelblue", mid = "white", high = "firebrick",
                       midpoint = 50, name = "% Gov") +
  theme_void() +
  labs(title = "Votação Média no Governo por Estado")
```




### 4.10.2 CRS — Sistema de Referência de Coordenadas

Todo dado geoespacial precisa de um CRS (Coordinate Reference System). Os dois mais comuns no Brasil:

- **EPSG:4326** — coordenadas geográficas (latitude/longitude), usado em mapas temáticos e GPS
- **EPSG:5880** — projeção métrica (metros), necessário para cálculos de distância e área

```r
# Verificar CRS de um objeto sf
st_crs(estados_br)
```

```
Coordinate Reference System:
  User input: SIRGAS 2000
  wkt:
GEOGCRS["SIRGAS 2000",
    DATUM["Sistema de Referencia Geocentrico para las AmericaS 2000",
        ELLIPSOID["GRS 1980",6378137,298.257222101,
            LENGTHUNIT["metre",1]]],
    PRIMEM["Greenwich",0,
        ANGLEUNIT["degree",0.0174532925199433]],
    CS[ellipsoidal,2],
        AXIS["geodetic latitude (Lat)",north,
            ORDER[1],
            ANGLEUNIT["degree",0.0174532925199433]],
        AXIS["geodetic longitude (Lon)",east,
            ORDER[2],
            ANGLEUNIT["degree",0.0174532925199433]],
    USAGE[
        SCOPE["Horizontal component of 3D system."],
        AREA["Latin America - Central America and South America - onshore and offshore. Br
        BBOX[-59.87,-122.19,32.72,-25.28]],
    ID["EPSG",4674]]
```

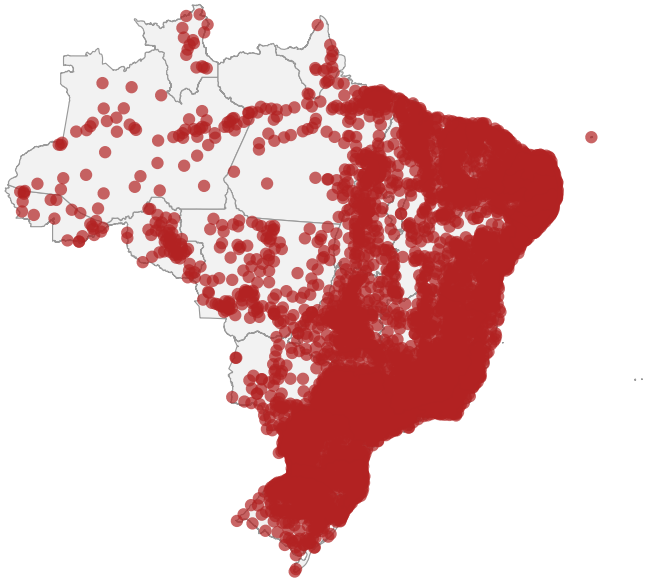
```r
# Reprojetar para CRS métrico (necessário para st_buffer, st_distance)
estados_metrico <- estados_br |>
  st_transform(5880)
```

### 4.10.3 Mapa com capitais — pontos sobre polígonos

```r
# Malha de municípios e capitais
capitais_geo <- read_municipal_seat(showProgress = FALSE)
```

```r
ggplot() +
  geom_sf(data = estados_br, fill = "grey95", color = "grey60") +
  geom_sf(data = capitais_geo, color = "firebrick", size = 1.5, alpha = 0.7) +
  theme_void() +
  labs(title = "Capitais Brasileiras")
```

Capitais Brasileiras



## 4.11 Pontos Ideais: análise espacial do comportamento político

### 4.11.1 A intuição

Se observarmos como parlamentares **votam** em diversas matérias, podemos inferir suas **posições ideológicas** em um espaço latente. O método mais influente para isso é o **DW-NOMINATE** (Poole & Rosenthal), que, a partir de centenas de votações nominais, estima onde cada parlamentar se localiza num eixo ideológico.

No contexto brasileiro, o primeiro eixo costuma capturar a dimensão **governo-oposição** (mais do que esquerda-direita), como demonstrado por Zucco & Lauderdale (2011).

Vamos implementar uma versão simplificada usando PCA (Análise de Componentes Principais) sobre uma matriz de votações simuladas.

### 4.11.2 Simulando votações nominais

```
set.seed(2025)
n_parl <- 40
n_vot <- 80

# Posições ideológicas "verdadeiras"
posicao_ideal <- c(
  rnorm(10, -1.5, 0.3),   # esquerda
```

```r
  rnorm(10, -0.3, 0.4),   # centro-esquerda
  rnorm(10,  0.5, 0.4),   # centro-direita
  rnorm(10,  1.5, 0.3)    # direita
)
bloco <- rep(c("Esquerda","Centro-Esq","Centro-Dir","Direita"), each = 10)

# Simular votos: probabilidade de votar Sim depende da posição vs. ponto de corte
pontos_corte <- runif(n_vot, -2, 2)
votos_mat <- matrix(NA, n_parl, n_vot)
for(j in 1:n_vot) {
  votos_mat[, j] <- rbinom(n_parl, 1, plogis(2 * (posicao_ideal - pontos_corte[j])))
}
```

A lógica é: cada votação tem um "ponto de corte" no eixo ideológico. Parlamentares cuja posição está acima do corte têm maior probabilidade de votar **Sim**. A função `plogis()` transforma a distância entre posição e corte em uma probabilidade entre 0 e 1.

### 4.11.3 Estimando pontos ideais via PCA

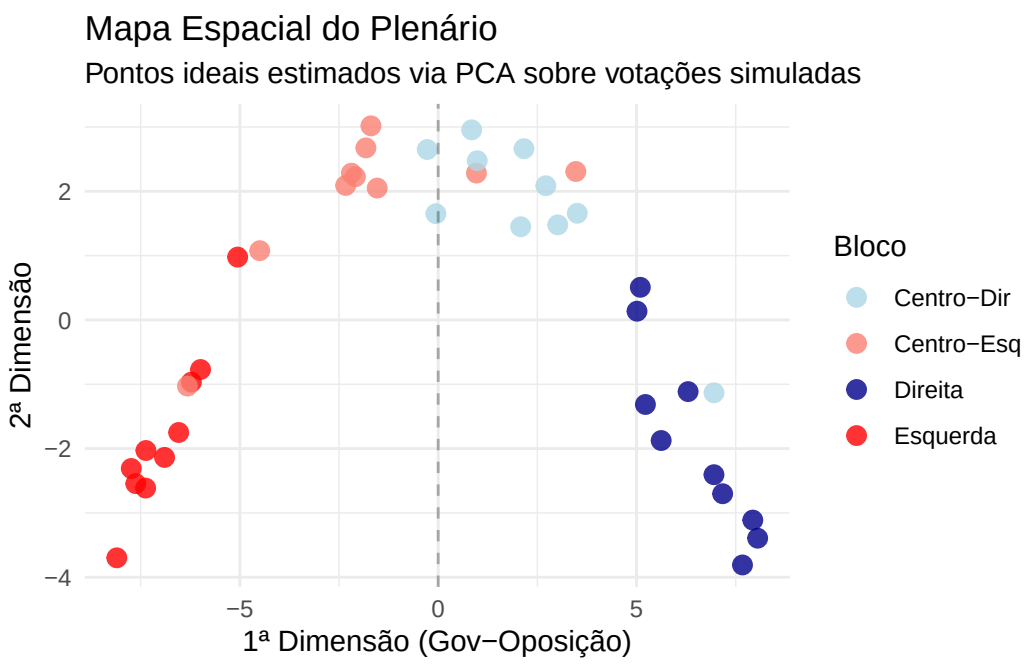
```r
pca <- prcomp(votos_mat, center = TRUE, scale. = TRUE)

pontos <- tibble(
  bloco = bloco,
  posicao_real = posicao_ideal,
  pc1 = pca$x[, 1],
  pc2 = pca$x[, 2]
)
```

### 4.11.4 Mapa do Plenário

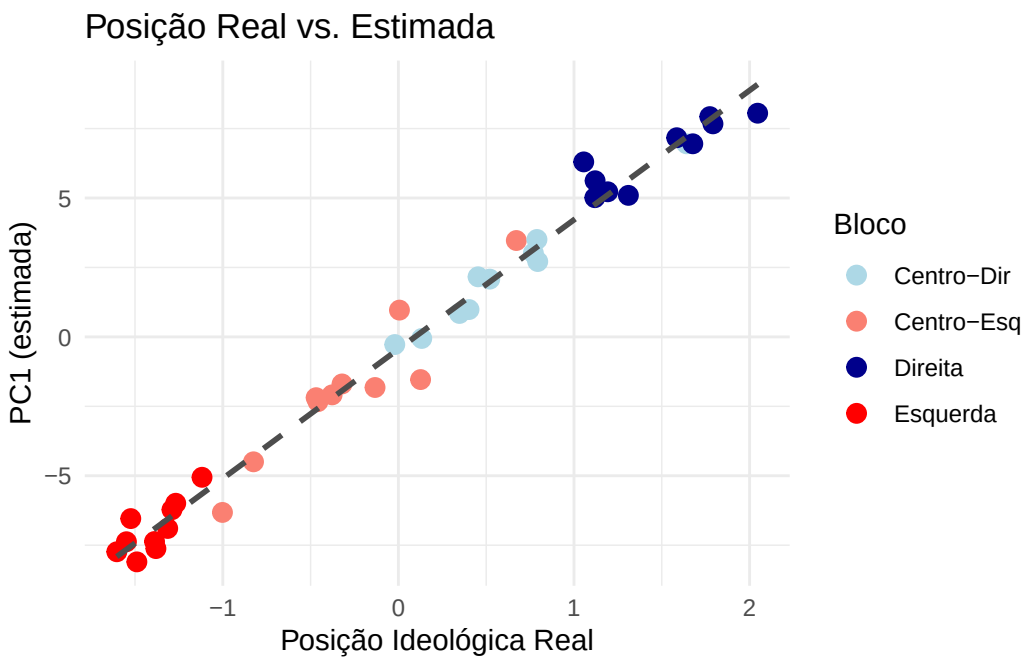
```r
ggplot(pontos, aes(x = pc1, y = pc2, color = bloco)) +
  geom_point(size = 3, alpha = 0.8) +
  scale_color_manual(values = c("Esquerda" = "red", "Centro-Esq" = "salmon",
                                "Centro-Dir" = "lightblue", "Direita" = "darkblue")) +
  geom_vline(xintercept = 0, linetype = "dashed", alpha = 0.3) +
  labs(title = "Mapa Espacial do Plenário",
       subtitle = "Pontos ideais estimados via PCA sobre votações simuladas",
```

```
      x = "1ª Dimensão (Gov-Oposição)", y = "2ª Dimensão", color = "Bloco") +
  theme_minimal()
```




### 4.11.5 Validação: posição real vs. estimada

```
ggplot(pontos, aes(x = posicao_real, y = pc1, color = bloco)) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = FALSE, color = "grey30", linetype = "dashed") +
  scale_color_manual(values = c("Esquerda" = "red", "Centro-Esq" = "salmon",
                                "Centro-Dir" = "lightblue", "Direita" = "darkblue")) +
  labs(title = "Posição Real vs. Estimada",
       x = "Posição Ideológica Real", y = "PC1 (estimada)", color = "Bloco") +
  theme_minimal()
```




A correlação entre as posições reais e estimadas demonstra que o PCA, mesmo sendo uma técnica simples, recupera bem a estrutura latente das votações.

### 4.11.6 Com dados reais: API da Câmara

Na prática, para estimar pontos ideais a partir de dados reais da Câmara dos Deputados, o procedimento envolve quatro passos:

```
# Passo 1: coletar votações de um período
url_base <- "https://dadosabertos.camara.leg.br/api/v2"

resp_votacoes <- request(paste0(url_base, "/votacoes")) |>
  req_url_query(dataInicio = "2024-08-01", dataFim = "2024-12-31",
                ordem = "ASC", ordenarPor = "dataHoraRegistro") |>
  req_perform()

lista_votacoes <- resp_votacoes |>
  resp_body_json() |>
  pluck("dados") |>
  map_chr(~ .x$id)

# Passo 2: para cada votação, coletar os votos individuais
coletar_votos <- function(id_votacao) {
  url <- paste0(url_base, "/votacoes/", id_votacao, "/votos")
  resp <- request(url) |> req_perform()
```

```r
  resp |>
    resp_body_json() |>
    pluck("dados") |>
    map_dfr(~ tibble(
      deputado = .x$deputado_$nome,
      partido  = .x$deputado_$siglaPartido,
      voto     = .x$tipoVoto
    )) |>
    mutate(id_votacao = id_votacao)
}

todos_votos <- map_dfr(lista_votacoes, coletar_votos)

# Passo 3: construir matriz deputado × votação
matriz_votos <- todos_votos |>
  mutate(voto_num = case_when(
    voto == "Sim" ~ 1,
    voto == "Não" ~ 0,
    TRUE ~ NA_real_
  )) |>
  select(deputado, id_votacao, voto_num) |>
  pivot_wider(names_from = id_votacao, values_from = voto_num) |>
  column_to_rownames("deputado")

# Passo 4: PCA nos votos (removendo NAs)
matriz_limpa <- matriz_votos |>
  select(where(~ mean(is.na(.x)) < 0.3)) |>
  drop_na()

pca_real <- prcomp(matriz_limpa, center = TRUE, scale. = TRUE)
```

**Alternativas ao PCA**

Para estimações mais sofisticadas de pontos ideais, existem pacotes dedicados como `wnominate` (implementação R do DW-NOMINATE) e `pscl` (ideal points bayesianos). Para o contexto brasileiro, a referência é Zucco & Lauderdale (2011), que aplicam modelos IRT (Item Response Theory) às votações da Câmara.

## 4.12 Exercícios — Aula 4

1. Crie um gráfico de dispersão de `taxa_comparecimento` (x) por `perc_votos_gov` (y), colorindo por `regiao`. Adicione `geom_smooth(method = "lm")` para cada região. O que você observa?

2. Crie um mapa coroplético do Brasil com o IDH médio por estado (use os dados de `municipios`). Escolha uma paleta de cores adequada.

3. Modifique o código de pontos ideais para simular 60 parlamentares em 3 blocos (governo, independente, oposição) e 120 votações. Plote o mapa do plenário e compare com a versão de 4 blocos.

4. Usando `patchwork`, combine o mapa do exercício 2 com o gráfico de barras por região em uma única composição.

---

# 5 Comunicando seus Dados (Aula 5)

## 5.1 Quarto: a evolução do R Markdown

Quarto é o sistema de publicação científica e técnica do ecossistema R/RStudio. Ele permite integrar código, texto e resultados em um único documento, gerando saídas em múltiplos formatos: HTML, PDF, Word, slides e até sites completos.

Estas slides e esta apostila foram feitos em Quarto.

### 5.1.1 Estrutura de um documento Quarto

Um arquivo `.qmd` tem três componentes:

1. **Cabeçalho YAML** (entre `---`): metadados do documento (título, autor, formato)
2. **Texto em Markdown**: prosa usando sintaxe simples de formatação
3. **Chunks de código** (entre ```` ```{r} ```` e ```` ``` ```): blocos de código R executáveis

```
---
title: "Meu Artigo"
author: "Seu Nome"
format: pdf
---

## Introdução

Texto aqui, com código integrado:
```

### 5.1.2 Formatos de saída

| Formato | Uso | Cabeçalho YAML |
|---|---|---|
| HTML | Web, blogs | `format: html` |
| PDF | Artigos, relatórios | `format: pdf` |
| Word | Fluxos Office | `format: docx` |
| revealjs | Apresentações | `format: revealjs` |

### 5.1.3 Opções de chunk

As opções de chunk controlam como cada bloco de código é processado. No Quarto, usam-se comentários especiais com `#|`:

```
#| echo: false          # esconde o código, mostra resultado
#| eval: false           # não executa o código
#| warning: false        # esconde avisos
#| fig-cap: "Legenda"    # legenda da figura
#| fig-width: 8          # largura da figura
#| fig-height: 5         # altura da figura
#| cache: true           # armazena resultado em cache
```

## 5.2 Tabelas com `knitr::kable()`

Para tabelas simples em documentos:

```r
municipios |>
  group_by(regiao) |>
  summarise(
    n = n(),
    idhm_medio = mean(idhm) |> round(3),
    voto_medio = mean(perc_votos_gov) |> round(1)
  ) |>
  knitr::kable(col.names = c("Região", "N", "IDHM Médio", "% Voto Gov"))
```

| Região | N | IDHM Médio | % Voto Gov |
|---|---|---|---|
| Centro-Oeste | 606 | 0.694 | 49.4 |
| Nordeste | 1794 | 0.601 | 59.1 |
| Norte | 311 | 0.631 | 51.0 |
| Sudeste | 1668 | 0.720 | 43.3 |
| Sul | 1191 | 0.733 | 42.7 |

## 5.3 Tabelas de regressão com `modelsummary`

O pacote `modelsummary` produz tabelas de regressão em formato acadêmico, compatíveis com os padrões de periódicos de ciência política:

Tabela 6: Determinantes do Voto no Governo

| | Bivariado | Controle Pop. | Completo |
|---|---|---|---|
| (Intercept) | 93.851*** | 95.053*** | 68.898*** |
| | (0.942) | (1.257) | (1.348) |
| idhm | −65.753*** | −65.736*** | −26.284*** |
| | (1.380) | (1.380) | (1.601) |
| log(populacao) | | −0.132 | −0.146+ |
| | | (0.091) | (0.080) |
| pib_per_capita | | | 0.000 |
| | | | (0.000) |
| regiaoNordeste | | | 7.290*** |
| | | | (0.406) |
| regiaoNorte | | | 0.017 |
| | | | (0.571) |
| regiaoSudeste | | | −5.333*** |
| | | | (0.384) |
| regiaoSul | | | −5.631*** |
| | | | (0.407) |
| Num.Obs. | 5570 | 5570 | 5570 |
| R2 | 0.290 | 0.290 | 0.456 |
| R2 Adj. | 0.289 | 0.290 | 0.455 |
| F | 2270.032 | 1136.282 | 665.210 |

+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001

```r
modelo1 <- lm(perc_votos_gov ~ idhm, data = municipios)
modelo2 <- lm(perc_votos_gov ~ idhm + log(populacao), data = municipios)
modelo3 <- lm(perc_votos_gov ~ idhm + log(populacao) + pib_per_capita + regiao,
              data = municipios)

modelsummary(
  list("Bivariado" = modelo1, "Controle Pop." = modelo2, "Completo" = modelo3),
  stars = TRUE,
  gof_omit = "AIC|BIC|Log|RMSE",
  title = "Determinantes do Voto no Governo"
)
```

> **Alternativas para tabelas**
>
> Além de `modelsummary`, o pacote `stargazer` é bastante usado na ciência política. A vantagem do `modelsummary` é sua compatibilidade nativa com Quarto e com diversos formatos de saída (HTML, LaTeX, Word).

## 5.4 Referências bibliográficas com BibTeX

Quarto permite citar referências bibliográficas usando arquivos `.bib`. No cabeçalho YAML:

```
---
title: "Meu Artigo"
bibliography: referencias.bib
csl: apsa.csl
---
```

No texto, use `@chave` para citações parentéticas e `[@chave]` para citações em texto. O Quarto gera automaticamente a lista de referências ao final do documento.

Exemplo de entrada no arquivo `.bib`:

```
@book{king1994,
  author    = {King, Gary and Keohane, Robert O. and Verba, Sidney},
  title     = {Designing Social Inquiry: Scientific Inference in Qualitative Research},
  publisher = {Princeton University Press},
  year      = {1994}
}
```

## 5.5 Reprodutibilidade: regras de ouro

A reprodutibilidade é um pilar da ciência. Em pesquisa quantitativa, isso significa que qualquer pessoa com acesso aos mesmos dados e código deve obter os mesmos resultados. Cinco práticas fundamentais:

1. **`set.seed()`** antes de qualquer operação com aleatoriedade — garante que simulações e amostras sejam replicáveis
2. **Caminhos relativos** — nunca use `/home/fred/...`; use `"dados/arquivo.csv"`
3. **RProjects** (`.Rproj`) — define automaticamente o diretório de trabalho
4. **`pacman::p_load()`** — centraliza o gerenciamento de pacotes no topo do script
5. **Quarto** — integra código e texto num fluxo único de trabalho

## 5.6 Organização de projetos

Uma boa estrutura de pastas é essencial para projetos reprodutíveis:

```
meu_projeto/
  meu_projeto.Rproj
  dados/
     brutos/
     processados/
  scripts/
     01_limpeza.R
     02_analise.R
     03_visualizacao.R
  resultados/
     figuras/
     tabelas/
  relatorio.qmd
```

> **O princípio RAP**
>
> RAP (Reproducible Analytical Pipelines) é a ideia de que toda análise deve funcionar como um pipeline: dados brutos entram, resultados finais saem, sem intervenção manual no meio. No R, ferramentas como `targets` automatizam esse fluxo.

## 5.7 Git e GitHub

Git é um sistema de controle de versão que registra todas as alterações feitas em seus arquivos. GitHub é a plataforma online que hospeda repositórios Git, permitindo backup, colaboração e publicação.

O RStudio tem integração nativa com Git — a aba "Git" no painel superior direito permite fazer commits, push e pull sem sair do IDE.

Para aprender a configurar: Happy Git with R

## 5.8 Próximos passos

Este curso cobriu os fundamentos. Para continuar aprendendo:

- **Shiny**: aplicações web interativas com R
- **tidymodels**: machine learning no tidyverse

- **quanteda**: análise de texto político
- **targets**: pipelines automatizados
- **R4DS** (R for Data Science, 2ª edição): https://r4ds.hadley.nz/

> **Dica**
>
> A melhor forma de consolidar o aprendizado é **aplicar R aos seus próprios dados de pesquisa**. Escolha um dataset relevante para seu tema e reproduza os exercícios deste curso com dados reais.

## 5.9 Exercícios — Aula 5

1. Crie um documento Quarto (`.qmd`) com formato PDF contendo: um título, um parágrafo de introdução, um chunk com código R que gere um gráfico, e um chunk com uma tabela `kable`. Renderize o documento.

2. Ajuste três modelos de regressão linear com a variável `perc_votos_gov` como dependente, adicionando variáveis progressivamente. Use `modelsummary()` para apresentá-los lado a lado.

3. Crie um arquivo `referencias.bib` com pelo menos 3 referências (use Google Scholar para obter entradas BibTeX). Cite-as em um documento Quarto.

4. Organize um mini-projeto de análise dos dados `municipios` seguindo a estrutura de pastas recomendada. O projeto deve incluir: um script de análise, um relatório Quarto, e uma pasta de resultados.

---

# Referências e Recursos

## 5.1 Livros e manuais


- Wickham, H., Çetinkaya-Rundel, M. & Grolemund, G. (2023). *R for Data Science* (2ª ed.). https://r4ds.hadley.nz/
- Wilke, C. (2019). *Fundamentals of Data Visualization*. https://clauswilke.com/dataviz/
- Chang, W. (2023). *R Graphics Cookbook* (2ª ed.). https://r-graphics.org/
- Tufte, E. (2001). *The Visual Display of Quantitative Information* (2ª ed.)


## 5.2 Referências em ciência política


- King, G., Keohane, R. O. & Verba, S. (1994). *Designing Social Inquiry*. Princeton University Press.
- Poole, K. T. & Rosenthal, H. (1997). *Congress: A Political-Economic History of Roll Call Voting*. Oxford University Press.
- Zucco, C. & Lauderdale, B. (2011). Distinguishing Between Influences on Brazilian Legislative Behavior. *Legislative Studies Quarterly*, 36(3), 363-396.


## 5.3 Links úteis

- Quarto: https://quarto.org/
- Posit Cheatsheets: https://posit.co/resources/cheatsheets/
- Stack Overflow (R): https://stackoverflow.com/questions/tagged/r
- API Câmara dos Deputados: https://dadosabertos.camara.leg.br/
- TSE Dados Abertos: https://dadosabertos.tse.jus.br/
- IBGE/SIDRA: https://sidra.ibge.gov.br/